

Data Structures in Java

Lecture 1: Course Overview.
Introduction to Data Structures and Algorithms.
Some Java Review.

09/07/2016

Daniel Bauer

Instructors

- Section 2:
 - Daniel Bauer <bauer@cs.columbia.edu>
 - Office hours: Wed 2:00pm-4:00pm
7LW3 CEPSR
- Section 1:
 - Paul Blaer <pblaer@cs.columbia.edu>
 - Office hours: Tue/Thu 2:00pm-4pm
703 CEPSR

Course Overview

- Lectures:
Section 2: Mon & Wed, 11:40am-12:55pm, 486 CSB
Section 1: Mon & Wed, 6:10pm - 7:25pm, 417 IAB
- Course info, lecture summaries, slides, code, gradebook, homework submission on (Courseworks 2 / Canvas)
- [piazza](#) used for questions / discussions / announcements.
- Make sure you can access all resources!

Instructional Assistants

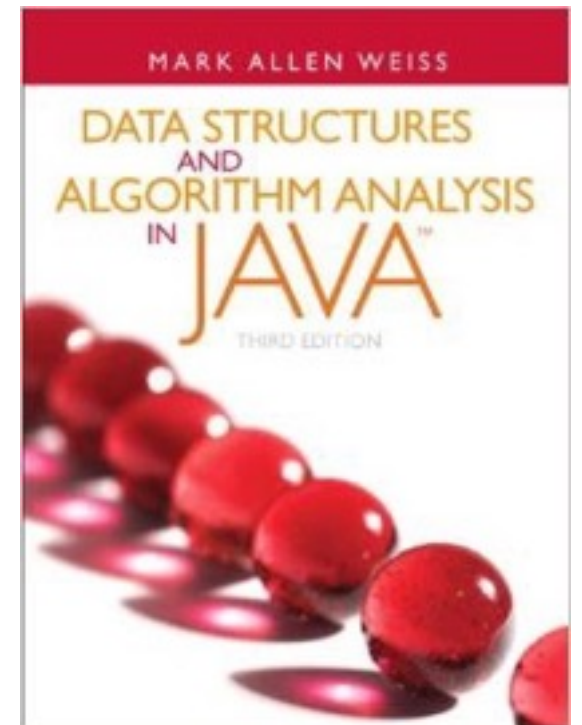
- Shared IA team for section 1 and 2.
- Head TA: Zeynep Ejder <ze2113@barnard.edu>
- IA hours, starting **next** week: Schedule will be announced on Canvas.
- There will be a Google poll to schedule recitation sessions.

Prerequisites

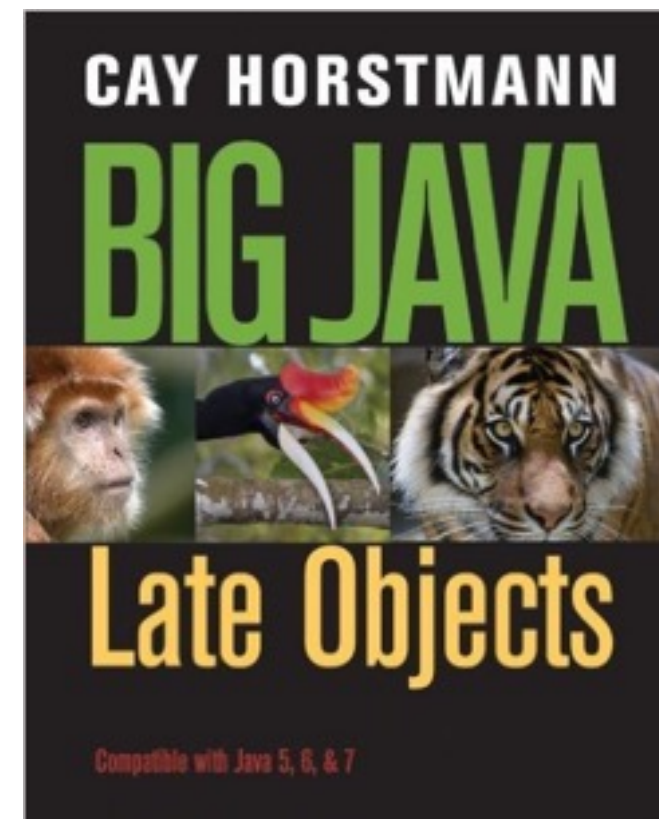
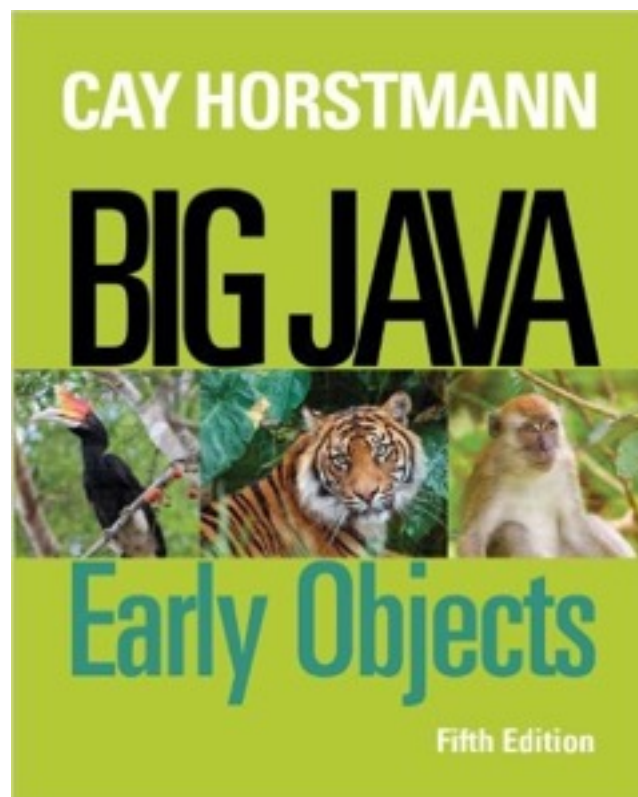
- **Knowledge of Java** (e.g. W1004 - Introduction to Computer Science and Programming in Java)
- Very basic math.
- Have some method for developing Java code - whatever works for you
 - Eclipse, IntelliJ, SublimeText, Emacs, Vim, ..
- Know how to compile and run Java code on the command line.

Textbook

- Weiss, Mark Allen (2012).
Data structures and Algorithm Analysis in Java.
3rd ed. Prentice Hall.
- ISBN: 9780132576277
- Errata:
<http://users.cis.fiu.edu/~weiss/dsaajava3/errata.html>
- Source code:
<http://users.cis.fiu.edu/~weiss/dsaajava3/code/>



Supplemental Books (Optional)



Horstmann, Cay (2013).
Big Java- Early Objects. 5th ed. Wiley.
ISBN: 1118431111

Deliverables and Grading

- 50% - 6 homework assignments, weakest dropped.
About two weeks each. Coding and theory.
- 20% - In-class midterm (October 26th).
- 30% - Final exam (TBA).
- Grading Range A+ to F

Homework 1

- Due Sep 23 by 4pm.
- Can submit until 11:55pm the following Tuesday (September 13) for a 20 point penalty.
- Follow submission instructions precisely.
- Start early, get help when you get stuck.

Expectations

- Attend class, participate!
- Start homework early.
- Get help when you need it (Piazza, office hours, IAs) and help others on Piazza.
- Make sure you have the registration status you want!
- Academic Honesty - read the statement on the Syllabus carefully. You **MUST** submit original work!

Why should I take Data Structures?

- Requirement, prerequisite for other CS courses.
- Core software engineering skills.
- Computational thinking: Develop problem solving skills using a “top-down” approach.
- Coding interviews usually focus on data structures and algorithms.

Digital Data



Text



Audio



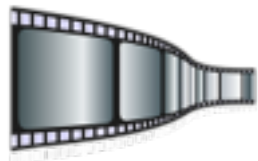
Social
Networks



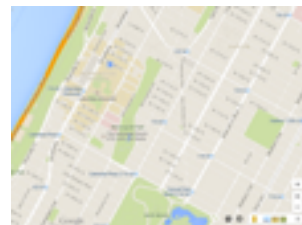
Tables



Images



Video



Maps



Bio data

...

Data Structures

- Data Structures are “containers” for data.
- They are used to store and organize data **so that they can be used efficiently.**
- Tasks: organize, search, filter, update, add, delete, combine

Example Applications

- Document Retrieval
- Route Planning
- Computer Graphics
- Natural Language Processing
- ...



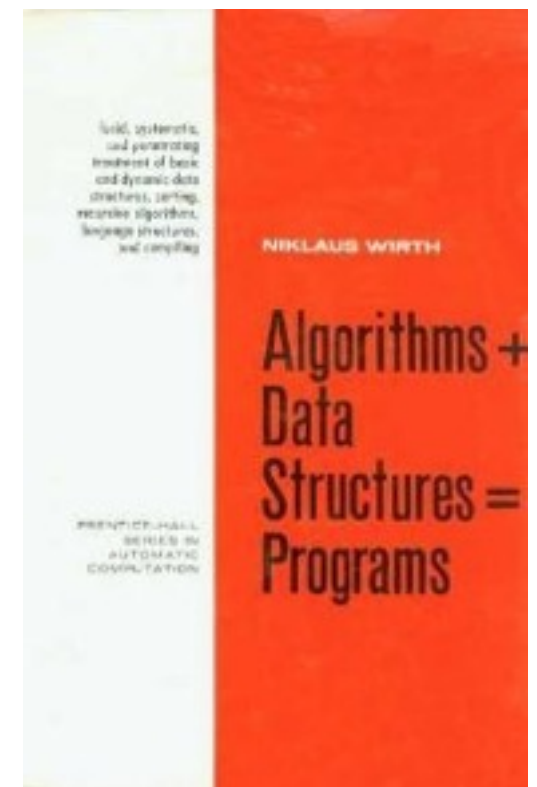
Big Data

- Ever faster hardware available, more memory.
- Amazon Web Services offers compute instances with 244GB of main memory!
- Can now store and process huge data sets.
- Ironically, algorithmic efficiency matters even more.



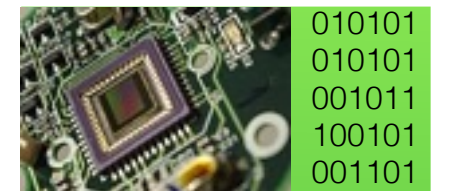
Data Structures + Algorithms = Programs

- Niklaus Wirth, 1976 (inventor of Pascal)
- Problem solving requires:
 1. Creating the right data model for thinking about a problem.
 2. Devising the appropriate methods to solve the problem.



Levels of Abstraction

- Hardware, bit representations
- Assembly language, registers, memory abstraction
- *Higher-level languages*
(Java, C++, Python, ...)
- Applications, User interfaces



```
L1: MV EDX, 00  
    INC EDX  
    CMP EDX, 10  
    JLE L1
```

```
public class Dag<E>  
    extends Graph<E> {...}
```



Course Overview

- Introduction to Algorithms. Recursion.
- Java review: OOP concepts, Inheritance, Interfaces, Generics.
- Abstract Data Types.
- Arrays, Lists.
- Trees, Search Trees. Sets and Maps.
- Priority Queues (Heaps)
- Sorting.
- Graphs.
- Introduction to NP-Completeness.

Arrays in Java

- Built-in data structure.
- Stores a sequence of objects of the same type.
- Fixed size.

```
int[] myArray;  
myArray = new int[10];  
  
myArray[0] = 0;  
myArray[1] = 10;  
myArray[2] = 5;
```

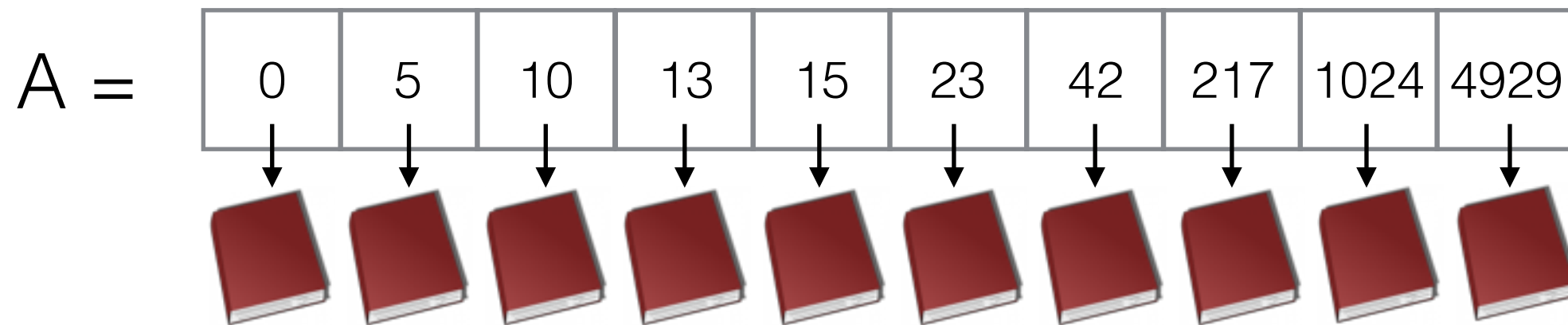
myArray:

0	5	10							
---	---	----	--	--	--	--	--	--	--

Algorithms

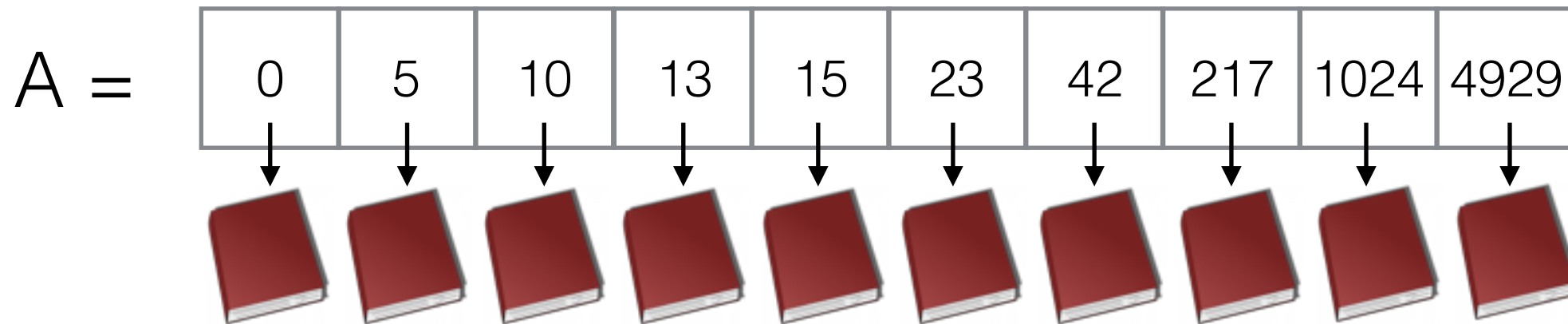
- An algorithm is a clearly specified set of simple instructions to be followed to solve a problem.
- Algorithm Analysis — Questions:
 - Does the algorithm terminate?
 - Does the algorithm solve the problem? (correctness)
 - What resources does the algorithm use?
 - Time / Space

Finding an Entry x in an Array



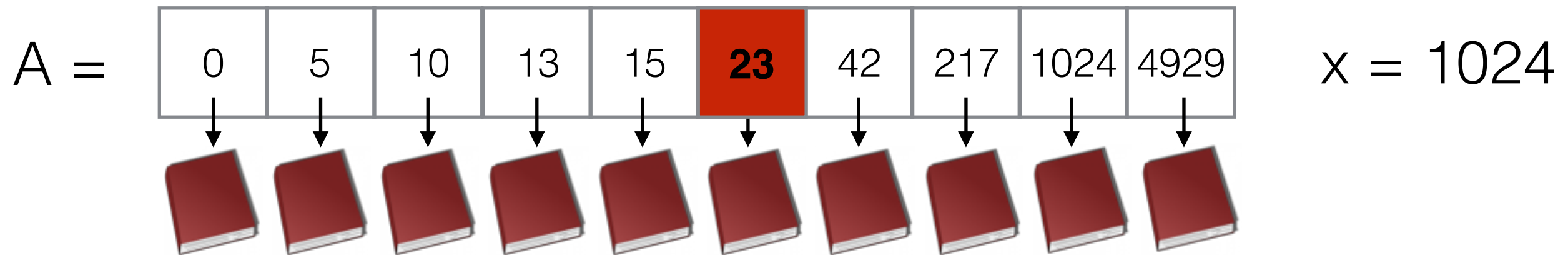
- Approach 1: Linear search. Start at position 0, scan list until we reach the correct entry.
- In the worst case, we need $\text{length}(A)$ steps to find the entry!

Finding an Entry x in a Sorted Array: Binary Search



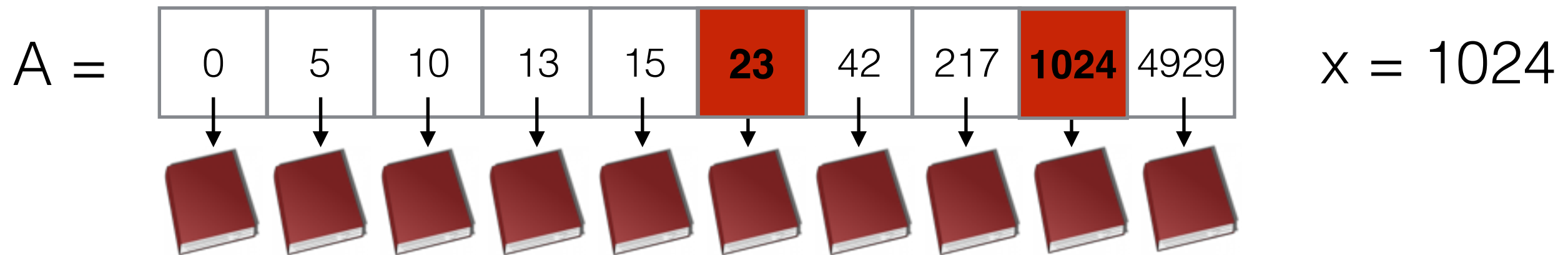
- Approach 2: **Use the property that the list is sorted.**
 - Find entry y in the middle of A : $y = A[A.length/2]$
 - if $(y == x)$ we found the entry.
 - if $(y < x)$ continue search on second half of A .
 - if $(y > x)$ continue search on first half of A .

Finding an Entry x in a Sorted Array: Binary Search



- Approach 2: **Use the property that the list is sorted.**
 - Find entry y in the middle of A : $y = A[A.length/2]$
 - if $(y == x)$ we found the entry.
 - if $(y < x)$ continue search on second half of A .
 - if $(y > x)$ continue search on first half of A .

Finding an Entry x in a Sorted Array: Binary Search



- Approach 2: **Use the property that the list is sorted.**
 - Find entry y in the middle of A : $y = A[A.length/2]$
 - if $(y == x)$ we found the entry.
 - if $(y < x)$ continue search on second half of A .
 - if $(y > x)$ continue search on first half of A .

Binary Search

```
public static int binarySearch(Integer[] a, Integer x) {  
  
    int low = 0;  
    int high = a.length - 1;  
    int mid;  
  
    while (low <= high) {  
        mid = (low + high) / 2;  
        if (a[mid] < x) { // x is in second half  
            low = mid + 1;  
        } else if (a[mid] > x) { // x is in first half  
            high = mid - 1;  
        } else {  
            return mid;  
        }  
    }  
    return -1;  
}
```

How many iterations of the *while* loop?

In the worst case we need $\log_2(\text{length}(a))$ steps.